# Generalidades a la hora de construir una clase

**Ing. Andrés Eduardo Paternina Ariza Esp.**

*Universidad del Magdalena*

apaternina@unimagdalena.edu.co

# Cómo leer el enunciado: pistas para cada frase

| Si el enunciado dice… | se construye… | En Vehiculo |
|---|---|---|
| «se identifica por», «se anotan» | Atributo privado y propiedad | _placa y Placa |
| «no se acepta», «nunca puede» (un dato) | if en el set que lanza excepción | Year, Kilometraje |
| «puede pedir», «registra», «puede cambiarla» | Método público | Cotizar, CambiarTarifaDiaria |
| «saber si…» | Propiedad bool calculada | EsAlquilable |
| «nadie más puede», «solo se puede» (una acción) | if en el método con mensaje | RegistrarAlquiler |
| «150.000 km», «más de una semana», «10 %» | Constante privada | LimiteKilometraje |

# Contexto: cada error tiene su defensa en la clase

*«Un mismo carro se le prometió a dos clientes al mismo tiempo»*

RegistrarAlquiler revisa el estado antes de entregarlo

```
if (Estado == EstadoAlquilado)
{
    Console.WriteLine("El vehículo ya esta alquilado");
    return;
}
```

*«Alguien corrigió a la baja el kilometraje»*

El set de Kilometraje es privado y solo deja avanzar

```
if (value < _kilometraje)
{
    throw new Exception("El kilometraje debe acumularse");
}
```

*«Carros con más de 150.000 km se siguieron alquilando»*

Una constante, un cambio automático de estado y un bloqueo

```
private const int LimiteKilometraje = 150000;
// AjustarEstadoSegunKilometraje() lo manda a
// mantenimiento y RegistrarAlquiler lo rechaza
```

# La placa: obligatoria y para toda la vida

Cómo funciona la flota, párrafo 1

**[1] Cada vehículo que entra a la flota** se identifica por su placa, **[2] que se le asigna al registrarlo y lo acompaña durante toda su vida en la empresa**; sin placa no hay forma de identificar el carro, así que **[3] ninguno entra a la flota sin ella**.

### [1] La clase y su constructor

Cada vehículo es un objeto; entrar a la flota es ejecutar el constructor.

```
public Vehiculo(string placa, string marca,
    string modelo, int year, double tarifaDiaria,
    int kilometrajeIngreso)
```

### [2] Placa con set privado y sin método para cambiarla

```
public string Placa
{
    get { return _placa; }
    private set { ... }
}
```

### [3] Validación en el set

```
if (string.IsNullOrWhiteSpace(value))
{
    throw new Exception("La placa es obligatoria");
}
```

# Los datos del registro y sus reglas

Cómo funciona la flota, párrafo 1

Al registrarlo también se anotan **[4] la marca, el modelo, el año de fabricación, la tarifa de alquiler por día y lo que marca el odómetro** en ese momento. Un odómetro empieza a contar desde cero, así que **[5] nunca puede marcar un valor negativo**. **[6] Tampoco se aceptan vehículos cuyo año de fabricación todavía no ha llegado**.

**[4] Atributos privados, uno por dato anotado**

```
private string _marca;
private string _modelo;
private int _year;
private double _tarifaDiaria;
private int _kilometraje;
```

**[5] Kilometraje no negativo**

```
if (value < 0)
{
    throw new Exception(
        "El kilometraje no puede ser negativo.");
}
```

**[6] Año no futuro**

```
if (value > DateTime.Now.Year)
{
    throw new Exception("El año no puede ser mayor...");
}
```

# La tarifa: la gerencia la cambia, la clase la protege

Cómo funciona la flota, párrafo 1

**[7] La tarifa diaria la fija la gerencia y puede cambiarla cuando lo considere necesario**, pero **[8] un alquiler gratuito o con tarifa negativa no tiene sentido para el negocio**.

**[7] Un método público para cambiarla**

```
public void CambiarTarifaDiaria(double nuevaTarifa)
{
    TarifaDiaria = nuevaTarifa;
}
```

**[8] La regla vive en el set privado**

```
private set
{
    if (value <= 0)
    {
        throw new Exception(
            "Debe tener una tarifa diaria mayor que cero.");
    }
    _tarifaDiaria = value;
}
```

# El estado inicial del vehículo

Cómo funciona la flota, párrafo 2

**[1] Un vehículo recién registrado queda listo para alquilarse**, **[2] a menos que ya llegue con más de 150.000 km: en ese caso va directamente a mantenimiento**. **[3] Todos los vehículos pertenecen a MoviCar**.

**[1] Estado inicial en el constructor**

```
Estado = EstadoDisponible;
```

**[2] Constante y ayuda privada, llamada al final del constructor**

```
private const int LimiteKilometraje = 150000;

private void AjustarEstadoSegunKilometraje()
{
    if (_kilometraje > LimiteKilometraje)
    {
        Estado = EstadoEnMantenimiento;
    }
}
```

**[3] Lo que NO se construyó**

Es igual para todos y ningún comportamiento depende de él. Guardarlo sería información innecesaria (criterio de abstracción).

# La cotización: un cálculo, no un dato guardado

**Cómo funciona la flota, párrafo 3**

Antes de decidirse, **[1] un cliente puede pedir una cotización indicando cuántos días necesita el vehículo**. **[2] El valor es la tarifa diaria multiplicada por los días solicitados**, y **[3] quienes alquilan por más de una semana reciben un descuento del 10 % sobre el total**.

**[1] Método que recibe los días y los valida**

```
public double Cotizar(int dias)
{
    if (dias <= 0) { ...; return 0; }
```

**[2] El cálculo base**

```
double total = _tarifaDiaria * dias;
```

**[3] Dos constantes y el descuento**

```
private const int DiasParaDescuento = 7;
private const double PorcentajeDescuento = 0.10;

if (dias > DiasParaDescuento)
{
    total = total - (total * PorcentajeDescuento);
}
```

# El alquiler: una entrega a la vez

Cómo funciona la flota, párrafo 3

**[1] Cuando el cliente confirma, el vehículo se le entrega y queda en su poder**; **[2] mientras tanto nadie más puede alquilarlo**. **[3] La empresa quiere saber en todo momento cuántas veces se ha alquilado cada vehículo**.

**[1] RegistrarAlquiler cambia el estado**

```
Estado = EstadoAlquilado;
```

**[2] Regla de la acción: va dentro del método**

```
if (Estado == EstadoEnMantenimiento) { ...; return; }
if (Estado == EstadoAlquilado) { ...; return; }
```

Cada if sale antes de tocar el estado: el vehículo queda igual que antes del intento.

**[3] Contador con set privado**

```
public int VecesAlquilado { get; private set; }

VecesAlquilado = VecesAlquilado + 1;
```

# La devolución: solo de un vehículo alquilado

**Cómo funciona la flota, párrafo 4**

Cuando el cliente regresa, **[1] el empleado revisa el odómetro y registra cuántos kilómetros recorrió durante el alquiler**. **[2] El odómetro de un carro solo avanza**. (...) Obviamente, **[3] solo se puede recibir de vuelta un vehículo que estaba alquilado**.

**[1] Método con los kilómetros recorridos**

```
public void RegistrarDevolucion(int kilometrosRecorridos)

Kilometraje = _kilometraje + kilometrosRecorridos;
```

**[2] Doble defensa del odómetro**

```
if (kilometrosRecorridos < 0) { ...; return; }
// y en el set: if (value < _kilometraje) throw ...
```

**[3] Primera regla del método**

```
if (Estado != EstadoAlquilado) { ...; return; }
```

# Después de la devolución: disponible o al taller

Cómo funciona la flota, párrafo 4

**[4] Si con ese recorrido el vehículo supera los 150.000 km, pasa automáticamente a mantenimiento**; **[5] si no, queda listo para el siguiente cliente**. (...) **[6] En esta primera versión no se gestiona la salida del taller: un vehículo que entra a mantenimiento permanece en ese estado**.

### [5] Primero vuelve a estar disponible

```
Estado = EstadoDisponible;
```

### [4] Luego la misma ayuda privada del constructor

```
AjustarEstadoSegunKilometraje();
```

"Automáticamente": nadie la llama desde afuera, la llama la propia clase.

### [6] Lo que no existe también es diseño

No hay método para sacarlo de mantenimiento y Estado tiene set privado. Por eso, una vez en el taller, no se puede volver a alquilar.

# Requisitos: cada viñeta tiene un miembro público

| La clase debe permitir… | Miembro de Vehiculo |
|---|---|
| Registrar un vehículo nuevo con sus datos | Constructor Vehiculo(...) |
| Cotizar un alquiler para una cantidad de días | Cotizar(int dias) |
| Registrar el alquiler | RegistrarAlquiler() |
| Registrar la devolución con los km recorridos | RegistrarDevolucion(int) |
| Saber si puede alquilarse y si necesita mantenimiento | EsAlquilable, NecesitaMantenimiento |
| Cambiar la tarifa diaria | CambiarTarifaDiaria(double) |
| Consultar datos, kilometraje, estado y veces alquilado | Getters públicos y VerInformacion() |

# «El vehículo debe quedar exactamente como estaba»: primero se valida, después se cambia

### Regla de un dato: en el set

```
private set
{
    if (value <= 0)
    {
        throw new Exception(...);
    }
    _tarifaDiaria = value;
}
```

La excepción sale antes de la asignación: el valor viejo nunca se pierde.

### Regla de una acción: en el método

```
if (Estado == EstadoAlquilado)
{
    Console.WriteLine(...);
    return;
}
Estado = EstadoAlquilado;
VecesAlquilado = VecesAlquilado + 1;
```

El return sale antes del primer cambio: ni el estado ni el contador se tocan.

# Abstracción: qué se guarda y qué no

### Se guarda

- Placa, marca, modelo
- Año
- Tarifa diaria
- Kilometraje
- Estado
- Veces alquilado

### Se calcula

- EsAlquilable
- NecesitaMantenimiento
- El valor de una cotización

### Regla fija

- Límite de 150.000 km
- 7 días para descuento
- 10 % de descuento
- Nombres de los estados

### No se guarda

- La empresa MoviCar
- El cliente
- Los días del alquiler

# Casos de prueba del taller

Vehículo ABC123, Toyota Corolla 2022, 145.000 km al ingresar, tarifa de $120.000 por día

| Caso | Acción | Resultado esperado | Lo decide |
|---|---|---|---|
| 1 | Cotizar 3 días y alquilar | $360.000. Alquilado, 1 alquiler | Cotizar, RegistrarAlquiler |
| 2 | Devolver con 2.000 km | 147.000 km. Disponible | RegistrarDevolucion |
| 3 | Cotizar 7 y 8 días | $840.000 y $864.000 | dias > DiasParaDescuento |
| 4 | Alquilar y devolver con 4.000 km | 151.000 km. En mantenimiento, 2 alquileres | AjustarEstado…() |
| 5 | Intentar otro alquiler | Rechazado con mensaje. Esta en mantenimiento | if EstadoEnMantenimiento |

# ¿Qué pasa cuando un dato pertenece al objeto, pero todavía no existe en el momento de crearlo?

# Caso 1: cuando una fecha todavía no existe

**Empleado.** De cada empleado se registra código, nombre, fecha de ingreso y fecha de retiro.
La fecha de retiro solo se conoce cuando el empleado deja la empresa.

### Malas salidas: fechas que significan algo

```
DateTime FechaRetiro = DateTime.Now;
DateTime FechaRetiro = new DateTime(1900, 1, 1);
```

Hoy y el 1 de enero de 1900 son fechas reales: ninguna dice «todavía no existe».

> **null no significa «vacío porque se me olvidó llenarlo».**
>
> Significa que en el dominio ese valor
> todavía no existe o no aplica.

### Buena salida: DateTime? que arranca en null

```
public DateTime? FechaRetiro { get; private set; }

// en el constructor
FechaRetiro = null;

public void RegistrarRetiro(DateTime fecha)
{
    FechaRetiro = fecha;
}
```

El ? permite null; con set privado, solo RegistrarRetiro asigna la fecha.

# Caso 2: cuando false no significa «no sabemos»

**Solicitud de crédito.** Se registra si el cliente fue aprobado.
Al crear la solicitud, todavía no ha sido evaluada.

### Mala salida: bool arranca en false

```
public bool Aprobada { get; set; }
// al crear la solicitud: Aprobada == false
```

false se lee como «fue evaluada y rechazada».

> Al principio queremos decir otra cosa:
>
> **«Todavía no sabemos si fue aprobada o rechazada.»**

### Buena salida: bool? con tres estados

```
public bool? Aprobada { get; private set; }

null   // todavía no ha sido evaluada
true   // aprobada
false  // rechazada
```

Aquí nullable sí tiene sentido: null es un estado real del trámite, «pendiente».

# ¿Uso nullable? Solo si «no tener valor» es un estado válido del objeto

**No preguntes:** «¿Este atributo es obligatorio?»

**Pregunta:** **«¿La ausencia de este valor tiene significado dentro del problema?»**

| Atributo | ¿Nullable? | Por qué |
|---|---|---|
| `string? SegundoNombre` | **Sí** | Una persona puede legítimamente no tener segundo nombre. |
| `DateTime? FechaRetiro` | **Sí** | Un empleado activo todavía no tiene fecha de retiro. |
| `bool? Aprobada` | **Sí** | Hay tres situaciones: pendiente, sí y no. |
| `double? Precio` | **No** | Todo producto debe tener precio: se exige en el constructor. |

# Tres preguntas rápidas: ¿nullable o no?

**`FechaDeNacimiento`**
de una persona

## Probablemente no

Si es obligatoria para registrarla,
se exige en el constructor.

**`FechaDeGraduacion`**
de quien todavía estudia

## Sí

Aún no se ha graduado:
la fecha todavía no existe.

**`TieneBeca`**
de un estudiante

## Depende

Se sabe desde el inicio:
`bool TieneBeca`

Pendiente de evaluación:
`bool? TieneBeca`

> **El tipo no lo decide C#. Lo decide el significado del dato en el problema.**

# Ejercicio 1: el empleado que se retira

**Enunciado**

De cada empleado se registra código, nombre y fecha de retiro.

La fecha de retiro solamente existe cuando el empleado abandona la empresa.

**Pregunta**

**¿Qué tipo tiene FechaRetiro?**

# Ejercicio 1: FechaRetiro es DateTime? porque al inicio no existe

**Empleado.** La fecha de retiro solamente existe cuando el empleado abandona la empresa.

### Solución: DateTime? que arranca en null

```
public DateTime? FechaRetiro { get; private set; }

public void RegistrarRetiro(DateTime fecha)
{
    FechaRetiro = fecha;
}
```

El ? permite null; solo RegistrarRetiro asigna la fecha.

### Cómo se lee

Mientras el empleado siga en la empresa:

**FechaRetiro == null significa «todavía no se ha retirado».**

# Ejercicio 2: el producto y su IVA

**Enunciado**

De cada producto se conoce precio y porcentaje de IVA.

El sistema debe permitir consultar el precio final.

**Pregunta**

**¿PrecioFinal es un atributo?**

# Ejercicio 2: el precio final se calcula, no se guarda

**Producto.** Se conoce precio y porcentaje de IVA; el sistema debe permitir consultar el precio final.

### Solución: una propiedad calculada

```
public double PrecioFinal
{
    get { return Precio * (1 + PorcentajeIva); }
}
```

PorcentajeIva se guarda como fracción: 0.19 para un IVA del 19 %.

### Mala salida: guardarlo como atributo

```
private double _precioFinal;

CambiarPrecio(50000);
// _precioFinal sigue con el valor viejo
```

Dos lugares con el mismo dato terminan contradiciéndose.

> Igual que la cotización de MoviCar:
>
> **un cálculo, no un dato guardado.**

# Ejercicio 3: la cuenta bancaria nueva

**Enunciado**

Una cuenta bancaria recién creada inicia activa y con cero transacciones.

**Pregunta**

**¿Qué sale de esta frase? ¿Son parámetros del constructor?**

# Ejercicio 3: un valor inicial no es un parámetro del constructor

**Cuenta bancaria.** Recién creada, inicia activa y con cero transacciones.

### Solución: la clase fija el valor inicial

```
public CuentaBancaria(string numero, string titular)
{
    Numero = numero;
    Titular = titular;

    Estado = "Activa";
    CantidadTransacciones = 0;
}
```

Número y titular son de ejemplo: cambian de una cuenta a otra.

### Mala salida: pedirlo por parámetro

```
new CuentaBancaria("001", "Ana", "Cerrada", 7);
```

Quien crea la cuenta podría arrancarla cerrada y con 7 transacciones.

> No todo lo que se asigna en el constructor llega por parámetro:
>
> **si es igual para todas las cuentas, lo fija la clase.**

# Ejercicio 4: las fechas de una reserva

**Enunciado**

Una reserva tiene fecha de inicio y fecha de finalización.

La fecha de finalización nunca puede ser anterior a la fecha de inicio.

**Pregunta**

**¿Contra qué compara el set de FechaFin?**

# Ejercicio 4: algunas validaciones dependen del estado del objeto

**Reserva.** La fecha de finalización nunca puede ser anterior a la fecha de inicio.

### Regla del dato: compara contra un valor fijo

```
if (value < 0)
{
    throw new Exception("No puede ser negativo");
}
```

Basta con mirar el valor nuevo.

Hay validaciones que dependen únicamente del dato

**y otras que dependen del estado del objeto.**

### Regla del objeto: compara contra otro atributo

```
public DateTime FechaFin
{
    get { return _fechaFin; }
    private set
    {
        if (value < FechaInicio)
        {
            throw new Exception(
                "El fin es anterior al inicio");
        }
        _fechaFin = value;
    }
}
```

En el constructor, FechaInicio se asigna antes que FechaFin.

# Ejercicio 5: los estados de un pedido

**Enunciado**

Un pedido solamente puede estar Pendiente, Pagado, Enviado o Cancelado.

**Pregunta**

## ¿Qué problema tiene esta propiedad?

```
public string Estado { get; set; }
```

# Ejercicio 5: un conjunto cerrado de valores se protege en la clase

**Pedido.** Con `public string Estado { get; set; }` nada impide escribir `pedido.Estado = "Batman";`

### Alternativa 1: constantes (valores fijos)

```
private const string EstadoPendiente = "Pendiente";
private const string EstadoPagado = "Pagado";
private const string EstadoEnviado = "Enviado";
private const string EstadoCancelado = "Cancelado";

public string Estado { get; private set; }

// cada método asigna una constante
Estado = EstadoPagado;
```

Como en MoviCar: set privado y solo los métodos cambian el estado.

### Alternativa 2: lista (valores que vienen de afuera)

```
private List<string> _estadosValidos;
public string Estado
{
    get { return _estado; }
    private set
    {
        if (!_estadosValidos.Contains(value))
        {
            throw new Exception("Estado no válido");
        }
        _estado = value;
    }
}
```

La lista llega al constructor, por ejemplo desde una base de datos.

# Ejemplo: paquete de mensajería

**Enunciado**

Una empresa de mensajería registra cada paquete con un código de guía, ciudad de destino, peso y fecha de recepción.

Cuando el paquete es creado en el sistema, todavía no se conoce la fecha exacta en la que será entregado al cliente. Esta información solo se registra cuando la entrega ocurre.

Además, algunos paquetes requieren una confirmación posterior para determinar si fueron recibidos personalmente por el destinatario. Mientras esa verificación no se haya realizado, el sistema no puede afirmar ni que sí ni que no fue recibido por él.

# Ejemplo: reserva de clase grupal

**Enunciado**

De cada reserva de una clase grupal se registra un código, nombre de la actividad, fecha de la clase, cantidad de cupos reservados y estado.

El código debe tener exactamente 6 caracteres y no puede cambiar después de creada la reserva. El nombre de la actividad es obligatorio, la cantidad de cupos debe estar entre 1 y 5 y no se pueden realizar reservas para fechas anteriores a la fecha actual.

Una reserva recién creada queda en estado “Pendiente”. Cuando se registra la reserva todavía no se conoce si el entrenador la aprobará. Posteriormente deberá poder registrarse esta decisión. Mientras no exista respuesta, el sistema no debe asumir que fue aprobada ni rechazada.

Si el entrenador la aprueba, el estado cambia a “Confirmada”. Si la rechaza, cambia a “Rechazada”. Una vez tomada la decisión, no puede modificarse.

La fecha real de asistencia tampoco se conoce cuando se crea la reserva y solo debe registrarse cuando el socio efectivamente asista.

Solo una reserva confirmada puede registrar asistencia. Al hacerlo, su estado cambia a “Finalizada”. Una reserva finalizada no puede modificarse nuevamente.

El sistema debe permitir consultar si la reserva todavía está esperando respuesta y si actualmente puede registrar asistencia.